

GH-200 • DAY 1 OF 2

# Automate your workflow with GitHub Actions

Today: how GitHub Actions really works, and every building block you need to write workflows you would put in front of a reviewer.

1 Fundamentals

2 Workflows

3 Variables and expressions

4 Artifacts, caching, concurrency

5 Deploying and reuse

DAY 1 AT A GLANCE

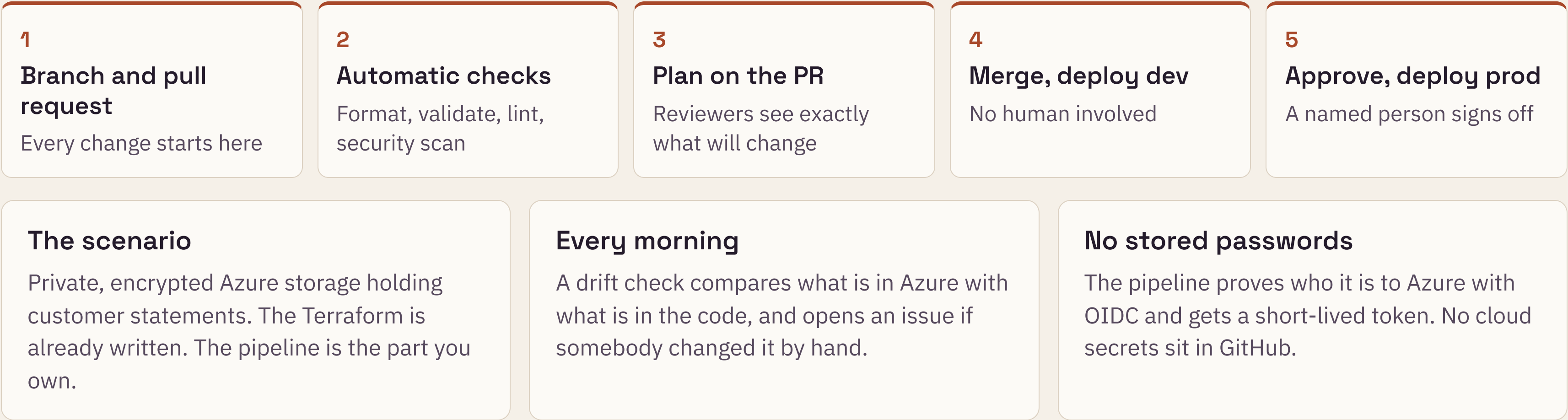
# Three parts, three labs, one plan for tomorrow

| Time  | Part                                  | What it covers                                                     | Lab   |
|-------|---------------------------------------|--------------------------------------------------------------------|-------|
| 09:00 | Why automate                          | Episode 1, and how the two days fit together                       |       |
| 09:30 | <b>1. Fundamentals</b>                | The six building blocks, your first workflow, using actions safely | Lab 1 |
| 11:15 | <b>2. Workflows, jobs and runners</b> | Triggers, jobs, matrices, runners, CI, and reading a red run       | Lab 2 |
| 14:00 | <b>3. Variables and contexts</b>      | Where a workflow gets its facts, and how it reacts to failure      | Lab 3 |
| 16:00 | Plan tomorrow                         | Draw the Terraform pipeline you will build on Day 2                |       |

Breaks at 11:00 and 15:30. Lunch 12:45 to 13:45. A Kahoot round at the end of every part. **Day 2** picks up with artifacts, caching, concurrency, environments and reuse, then the pipeline.

WHERE THIS IS GOING BY THE END OF DAY 2

# One change, from a laptop to production



## HOW WE WORK

# Four rules for two days

### Teams of 3 to 5

One person types, everybody else reads and checks. Swap the typist every single lab. No passengers.

### Always a pull request

Even for a one-line change. That is the habit this whole course is building, and tomorrow it is the only way in.

### Try first, then peek

Finished files are in the reference folder. Give yourselves ten minutes before you open them.

### Break things on purpose

A red run teaches more than a green one. Read the log before you put a hand up.

**And the fifth rule** The Kahoot runs all day, one round per part. Scores carry over. Nothing is on it that we have not covered.

# Every change built, tested and shipped the same way

## Manual is slow

Every release waits for a person to build, test and copy files. Queues form, releases get bigger, and bigger releases are riskier.

## Manual is uneven

Steps get skipped on busy days. Two people do the same task two different ways. Nobody can prove afterwards what actually ran.

## Automated is repeatable

The same steps run on every change, in the same order, with a log of who, what and when. That log is the audit trail.

## Continuous integration

Merge small changes often. Build and test each one automatically so problems show up in minutes, not at the end of a sprint.

## Continuous delivery

Every change that passes is ready to release. Deploying becomes a routine gated step instead of an event with a bridge call.

## GitHub Actions

The engine that runs all of this, sitting right next to your code, triggered by what happens in the repo.

PART 1 • 09:30

# GitHub Actions fundamentals

The six building blocks, your first workflow, and how to use somebody else's action without handing them your repository.

---

## YOU WILL BE ABLE TO

- Name events, workflows, jobs, steps, actions and runners
- Create and run the official quickstart workflow
- Tell apart GitHub Actions and GitHub Apps
- Reference an action so it cannot change under you

## WATCH AND READ

- Episode 2: Introduction to GitHub Actions, 47 min
- Docs: Understanding GitHub Actions
- Docs: Quickstart for GitHub Actions
- Docs: GitHub Actions vs GitHub Apps

# When something happens, run these instructions

**EVENT**

**Something happens**

A push, a pull request, a schedule, somebody clicking a button

**WORKFLOW**

**A YAML file**

Lives in `.github/workflows`. One file, one job to be done

**JOB**

**Runs on one runner**

Jobs run in parallel unless you say otherwise

**STEP**

**One command or action**

Steps run in order, top to bottom, on the same machine

**Runner**

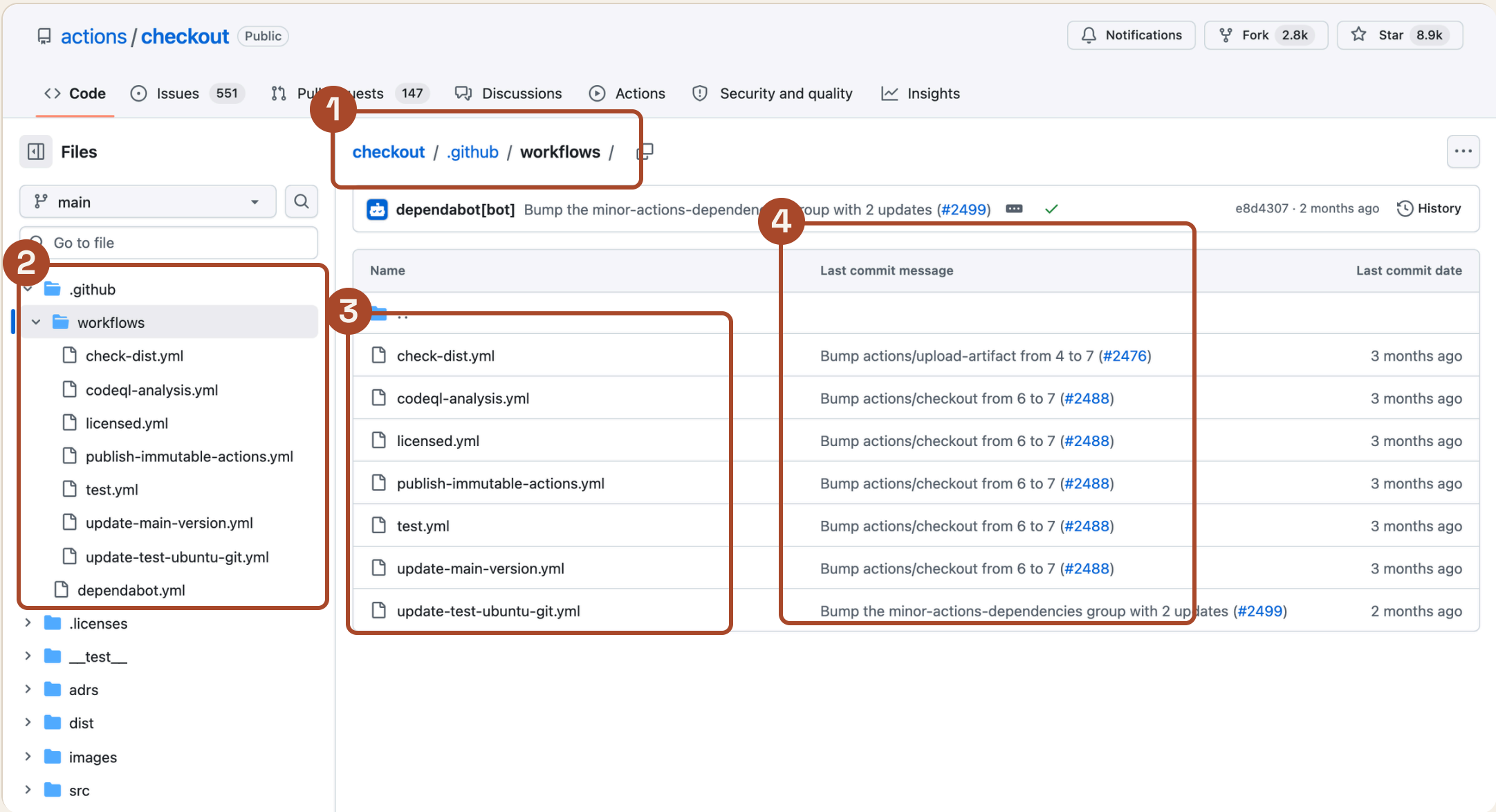
The machine that runs a job. GitHub hands you a fresh virtual machine, runs your job on it, then throws it away. Nothing survives to the next job.

**Action**

A reusable step that somebody has already packaged, like checking out your code or installing Terraform. You call it with **uses:**.

The whole thing in one sentence: an **event** starts a **workflow**, which runs **jobs** on **runners**, and each job is a list of **steps** that are either a command or an **action**.

# A workflow is just a file in your repository



- 1 The path is the rule**  
.github/workflows at the root of the repo.  
Anywhere else and GitHub never looks at it.
- 2 One folder, many files**  
A repo can hold as many workflows as you like.  
One per job to be done reads far better than one giant file.
- 3 Name them for what they do**  
test.yml, check-dist.yml, codeql-analysis.yml.  
Somebody debugging at 6pm should not have to open the file to guess.
- 4 They version like code**  
They sit in Git, they show in the diff, they need a review. That is the whole point.

Screenshot: github.com/actions/checkout, captured this morning.

# The official first workflow, line by line

```
name: GitHub Actions Demo
run-name: ${{ github.actor }} is testing out GitHub Actions
on: [push]
jobs:
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Triggered by a ${{ github.event_name }} event."
      - run: echo "Running on ${{ runner.os }}, hosted by GitHub."
      - name: Check out repository code
        uses: actions/checkout@<commit SHA> # v7.0.1
      - name: List files in the repository
        run: ls ${{ github.workspace }}
      - run: echo "This job's status is ${{ job.status }}."
```

## run-name

Sets the title of each run in the Actions tab. Worth setting, it makes the run list readable.

## on: [push]

Runs on every push to any branch. Fine for learning, far too broad for real work.

## run: versus uses:

run is a shell command on the runner. uses: pulls in somebody else's action.

## Nothing is there yet

Until checkout runs, the runner has none of your files. It is an empty machine.

## \${{ }}

Anything inside the double braces is an expression. GitHub fills it in before the step runs.

TASK 1 • 3 MIN

# Two jobs, one file, one trap

In your team. Ninety seconds to decide, then I take answers from two teams.

```
on: [push]
jobs:
  one:
    runs-on: ubuntu-latest
    steps:
      - run: echo "A" > note.txt
      - run: cat note.txt
  two:
    runs-on: ubuntu-latest
    steps:
      - run: cat note.txt
```

## Question 1

Does job **two** print A, print nothing, or fail?

## Question 2

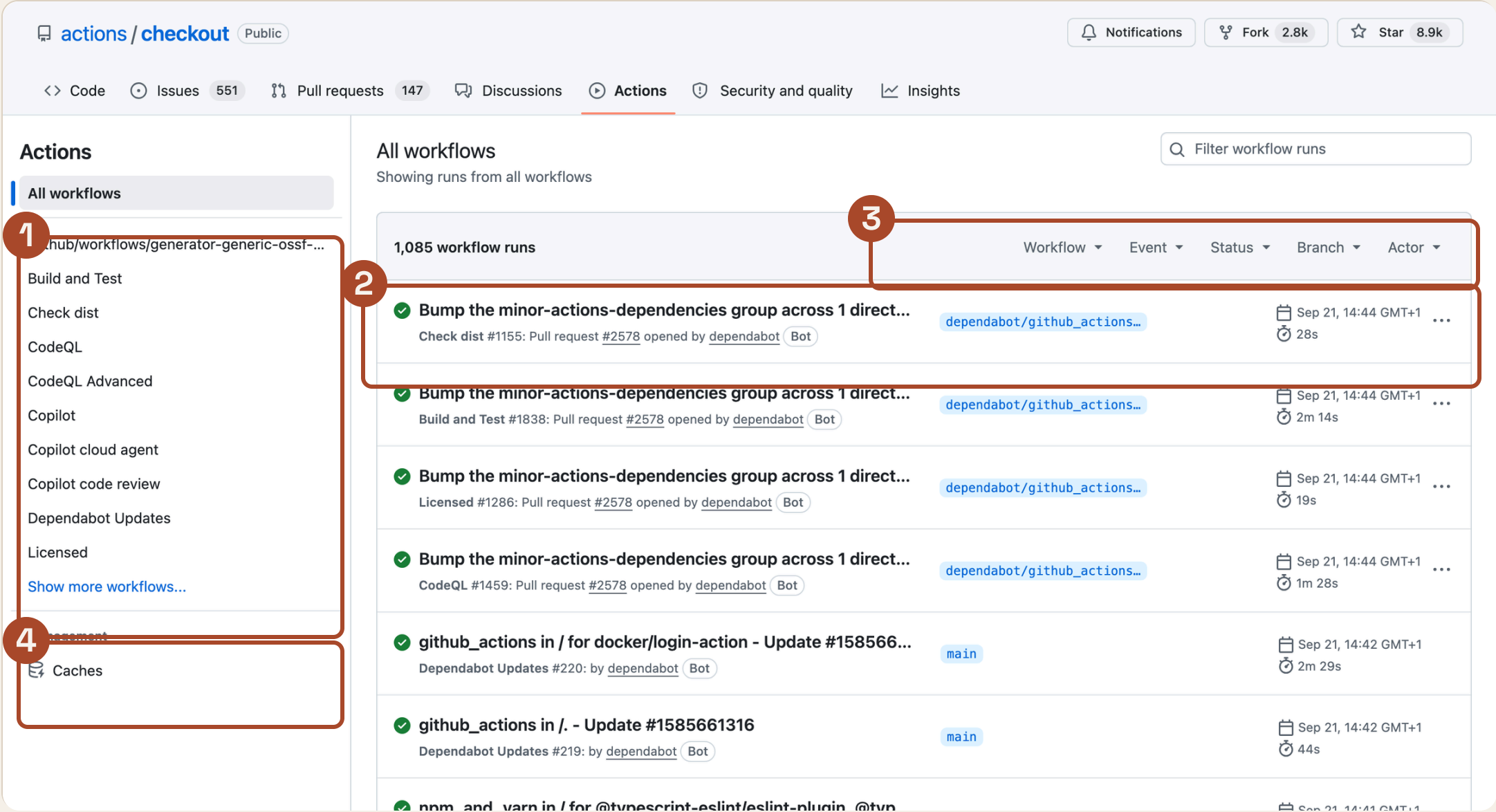
In what order do **one** and **two** run?

## Question 3

Change one line so **two** always runs after **one**. Which line?

Answers in the facilitator notes. Do not go past this slide until somebody has said the words "different machine".

# Where every run of every workflow shows up



- 1 One entry per workflow file**  
The sidebar lists your workflows by name. Click one to see only its runs.
- 2 Every run, newest first**  
Each row tells you the workflow, the event that started it, who or what triggered it, and how long it took.
- 3 Filter when it gets busy**  
By workflow, event, status, branch or actor. This repo has 1,085 runs. You will need this.
- 4 Caches live here too**  
Under Management. You can see what is cached and delete entries. We use this in Part 4.

The click path the docs teach: **Actions tab** → **the workflow in the sidebar** → **the run** → **the job** → **the step**. Five clicks from anywhere to a log line.

# Other people’s code runs in your pipeline

| How you reference it | What it looks like                   | Can it change under you?                              |
|----------------------|--------------------------------------|-------------------------------------------------------|
| A branch             | actions/checkout@main                | <b>Yes</b> , with every commit to that branch         |
| A tag                | actions/checkout@v7                  | <b>Yes</b> , a tag can be moved to a different commit |
| A full commit SHA    | actions/checkout@3d3c42e5aac5ba80... | <b>No</b> . This is what we use.                      |

### Why this matters here

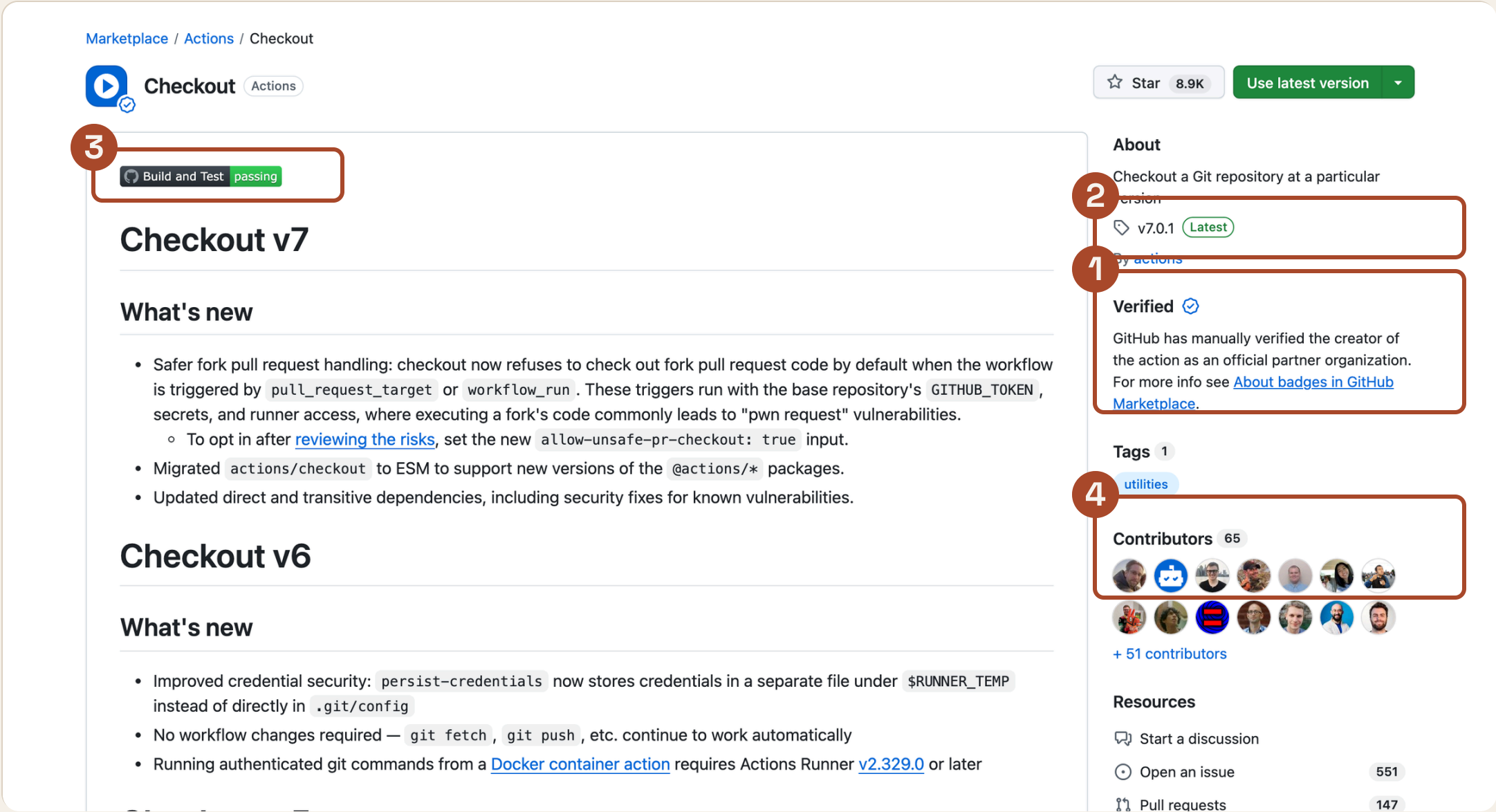
That action runs on the same machine as your code, with your workflow's token. If somebody moves the tag it points at, you run their new code on your next build and nobody opened a pull request.

### What good looks like

Pin to the full 40 character SHA, put the human version in a comment, and let Dependabot raise a pull request when there is a new release.

Keep the version readable: `uses: actions/checkout@3d3c42e5... # v7.0.1`

# What to look at before you type uses:



- 1 Verified creator**  
The badge means GitHub checked the publisher is who they claim to be. It is not a security audit, but an unverified one-star action deserves a second look.
- 2 A recent release**  
v7.0.1, marked Latest. An action whose last release was three years ago is a maintenance risk, not a bargain.
- 3 Its own build is passing**  
The action's own CI badge. If the maintainer's tests are red, that tells you something.
- 4 Real people use it**  
Contributors, issues, pull requests. Sixty-five contributors and an active issue tracker is a living project.

Screenshot: the Marketplace listing for actions/checkout.

# Two ways to automate GitHub. Pick by the job.

## GitHub Actions

- Runs on a runner, or inside a container
- Gets a clone of your repository
- No server of your own to deploy or keep alive
- Simple secrets for talking to other services
- Built for CI/CD and repository automation

## GitHub Apps

- Runs all the time and reacts to events fast
- Good when you must keep data between events
- Best for quick API calls, not long jobs
- Needs a server or compute that you provide
- Built for integrations and bots

Rule of thumb: a task that **starts, finishes and needs your code** is an Action. A service that **listens and remembers** is an App.

# Round 1: Fundamentals

1. Which folder does GitHub read workflows from?
2. Two jobs in one workflow. Does the second see files the first created?
3. What does `uses:` actually do?
4. Which reference cannot change under you: a branch, a tag or a SHA?

# Your first workflows

## GOAL

Run the official quickstart, then write a workflow that you start by hand.

## DONE WHEN

Both workflows have a green run, and you can say out loud what each context printed and why.

1. Create **github-actions-demo.yml** from the Quickstart, commit it on a branch, open a pull request
2. Open the **Actions** tab, open the run, and read every single step's log
3. Create **01-hello.yml** with **workflow\_dispatch** inputs: a name, and a boolean called shout
4. Click **Run workflow**, type a name, tick shout, and read the output
5. Change one **uses:** line to a pinned SHA. Re-run. Confirm it still works

PART 2 • 11:00

# Workflows and pipelines

How GitHub decides to start a run, how the file is laid out, and how jobs, steps and runners become a pipeline that guards your main branch.

---

## YOU WILL BE ABLE TO

- Explain exactly how GitHub decides to start a run
- Write triggers, job dependencies and a matrix
- Choose a runner on purpose, not by habit
- Build a CI workflow and read its logs when it goes red

## WATCH AND READ

- Episode 3: Pipelines in GitHub Actions, 67 min
- Docs: Workflows
- Docs: Continuous integration
- Docs: Choosing the runner for a job

# From an event to a running workflow

1

## An event happens

A push, a pull request, a schedule firing, somebody clicking Run

2

## It carries a commit

Every event brings a SHA and a Git ref along with it

3

## GitHub looks in the repo

It reads `.github/workflows` **at that commit**

4

## Matching workflows run

Every workflow whose **on:** matches gets a run

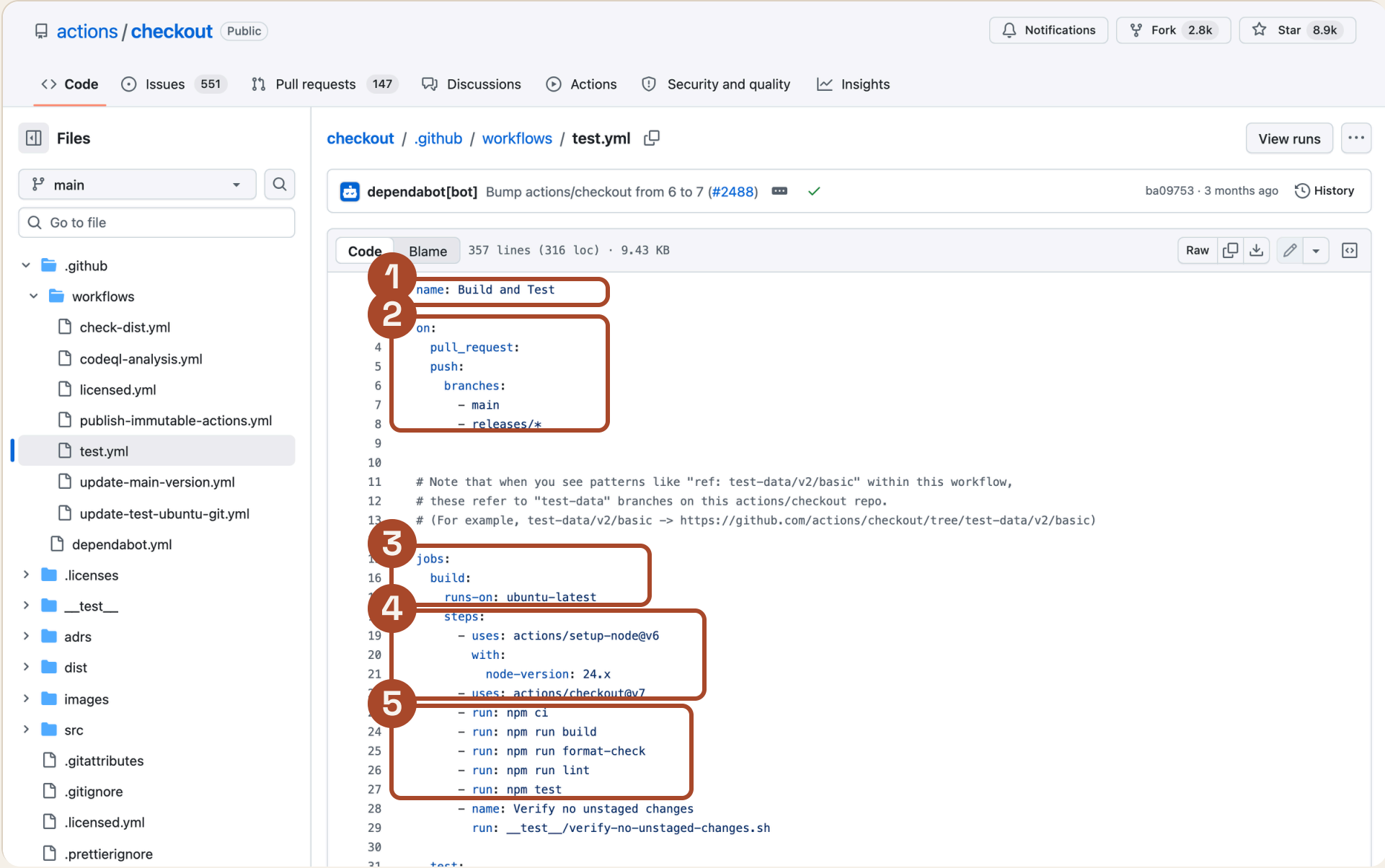
## Why step 3 is the one that catches people

The workflow file is read from the commit that triggered the event. Add a workflow on a branch and it only runs for that branch until you merge it. That is why your **Run workflow** button was missing in Lab 1.

## GITHUB\_SHA and GITHUB\_REF

Set on the runner for every run, so your steps always know exactly which commit and which branch they are working on. No guessing, no `git rev-parse`.

# Reading a real workflow file, top to bottom



- 1 name:**  
What shows in the Actions tab sidebar. Not the file name.
- 2 on:**  
The events. Here, pull\_request and a push to main or a releases branch. This is your blast radius.
- 3 jobs: and runs-on:**  
Each job names the machine it wants. One job, one runner, one fresh VM.
- 4 steps: with uses:**  
setup-node pulls in a packaged action. Notice the with: block underneath passing it an input.
- 5 steps: with run:**  
npm ci, npm run build, npm test. Plain shell on the runner, in order.

Screenshot: .github/workflows/test.yml in actions/checkout.

# Ways to start a workflow

| Event               | Starts when                            | Reach for it when                       |
|---------------------|----------------------------------------|-----------------------------------------|
| push                | Commits land on a branch or a tag      | Deploy after a merge to main            |
| pull_request        | A PR opens, or gets new commits        | Checks that run before anyone can merge |
| workflow_dispatch   | Somebody clicks Run, with typed inputs | Manual jobs, reruns, one-off reports    |
| schedule            | A cron time, always in UTC             | Nightly drift checks, daily reports     |
| repository_dispatch | Another system calls the GitHub API    | Triggers from outside GitHub            |
| workflow_call       | Another workflow calls this one        | Reusable workflows, which is Part 5     |

Narrow it with **branches**, **paths** and **types** so a workflow only runs when it actually matters. **paths** is the one nobody uses and everybody needs.

# workflow\_dispatch, with typed inputs

```
on:
  workflow_dispatch:
    inputs:
      name:
        type: string
        required: true
        default: Lagos
      shout:
        type: boolean
        default: false
      target:
        type: environment # from your environments
```

## It becomes a form

Inputs show up as fields on the Run workflow button, and as the **inputs** context inside the workflow.

## Five types

string, boolean, number, choice for a fixed list, and environment, which reads the environments you have defined.

## The button lives on the default branch

GitHub only shows Run workflow once the file is on your default branch. This bit them in Lab 1.

## You will see this block again

The same inputs: block under **workflow\_call** is how reusable workflows take parameters. Part 5.

# Match the job to the trigger

Five things you actually want. Two minutes in your team, then I go round the room.

| The thing you want                               | Which trigger |
|--------------------------------------------------|---------------|
| A pull request must be checked before merge      | ?             |
| Deploy to dev the moment a change lands on main  | ?             |
| Compare Azure with the code every morning at 6   | ?             |
| An ops person needs to restore staging on demand | ?             |
| A central repo wants every team to reuse one job | ?             |

Pick from

push · pull\_request · workflow\_dispatch ·  
schedule · repository\_dispatch · workflow\_call

Then the harder half

For the first one, what stops somebody merging  
anyway while the check is still red?

The second question is the one that matters. It is not a trigger at all.

# In parallel, in order, or once per variation

```
jobs:
  check: # these two start
    runs-on: ubuntu-latest
  scan: # at the same time
    runs-on: ubuntu-latest
  plan:
    needs: [check, scan] # waits for both
    runs-on: ubuntu-latest
  drift:
    strategy:
      fail-fast: false
    matrix:
      environment: [dev, prod]
```

## needs

Makes a job wait. If a job it needs fails, this one is skipped, not failed. Read your run summary carefully.

## matrix

Turns one job into several: per OS, per version, per environment. Add include, exclude and max-parallel to shape it.

## fail-fast: false

By default one red matrix leg cancels the rest. Turn it off when you want to see every failure, not just the first.

## if

Run a job or a step only when a condition holds, like **github.ref == 'refs/heads/main'**. Part 3.

# Where your jobs actually run

## GitHub-hosted

A fresh VM per job, patched and thrown away by GitHub. **ubuntu-latest** is currently Ubuntu 24.04. Pin to **ubuntu-24.04** when a surprise upgrade would hurt you.

## Larger runners

Still GitHub-hosted, with more CPU and memory, fixed IP ranges and Azure private networking. This is usually the answer for a bank that needs network controls.

## Self-hosted

Your own machine or VM. Full control of the network and the tooling, and full responsibility for patching and cleanup. Never put one on a public repo.

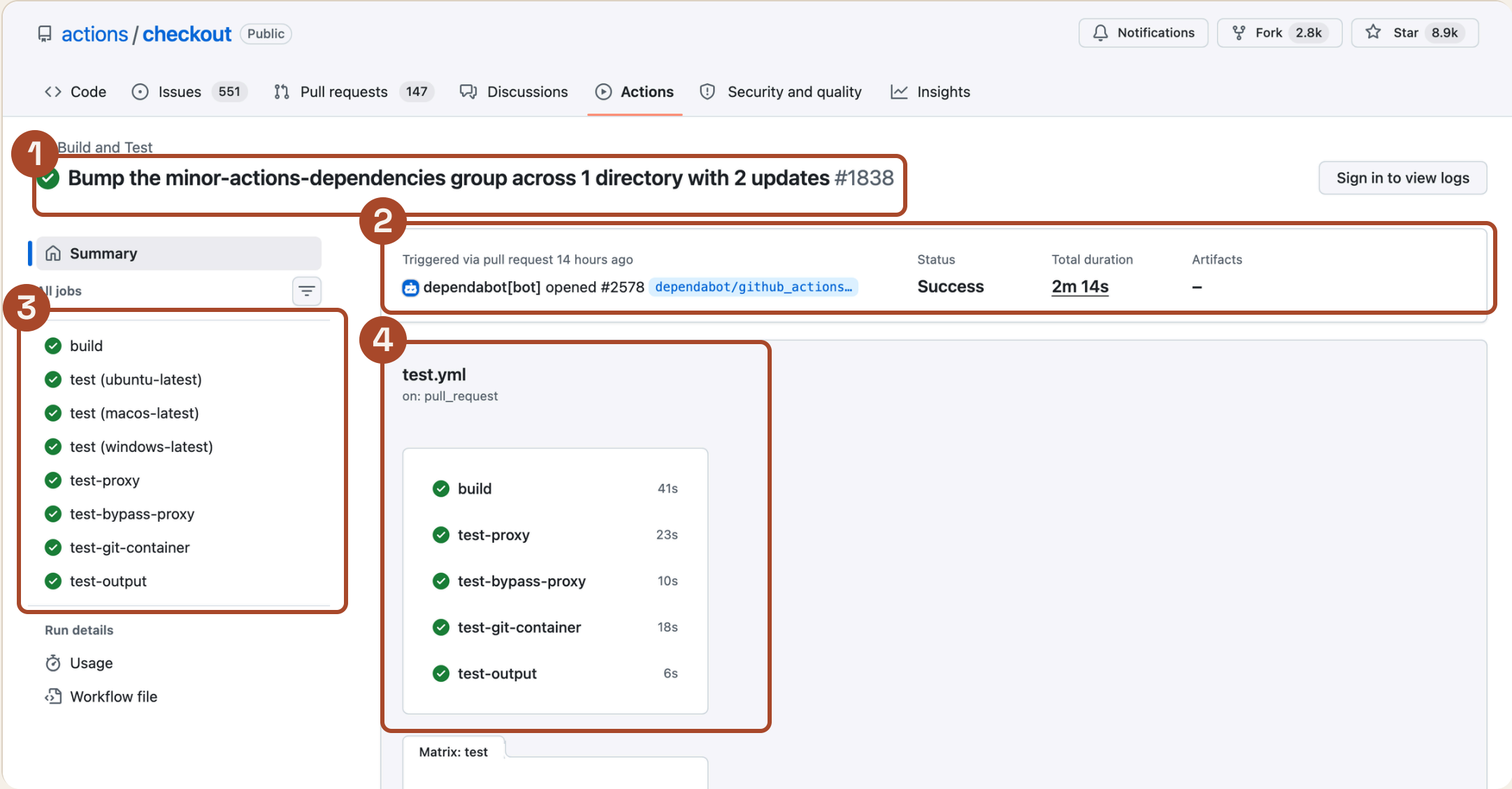
## What is already installed

Each runner image ships with a long list of tools, and each image's release notes list them exactly. Anything missing: install it with a **setup-\*** action, a package manager, or run the job in a container.

## The label is a promise you did not make

**latest** means GitHub moves it when they are ready. Ubuntu 20.04 is gone. windows-latest has moved to Server 2025. If your build is fragile, pin the version.

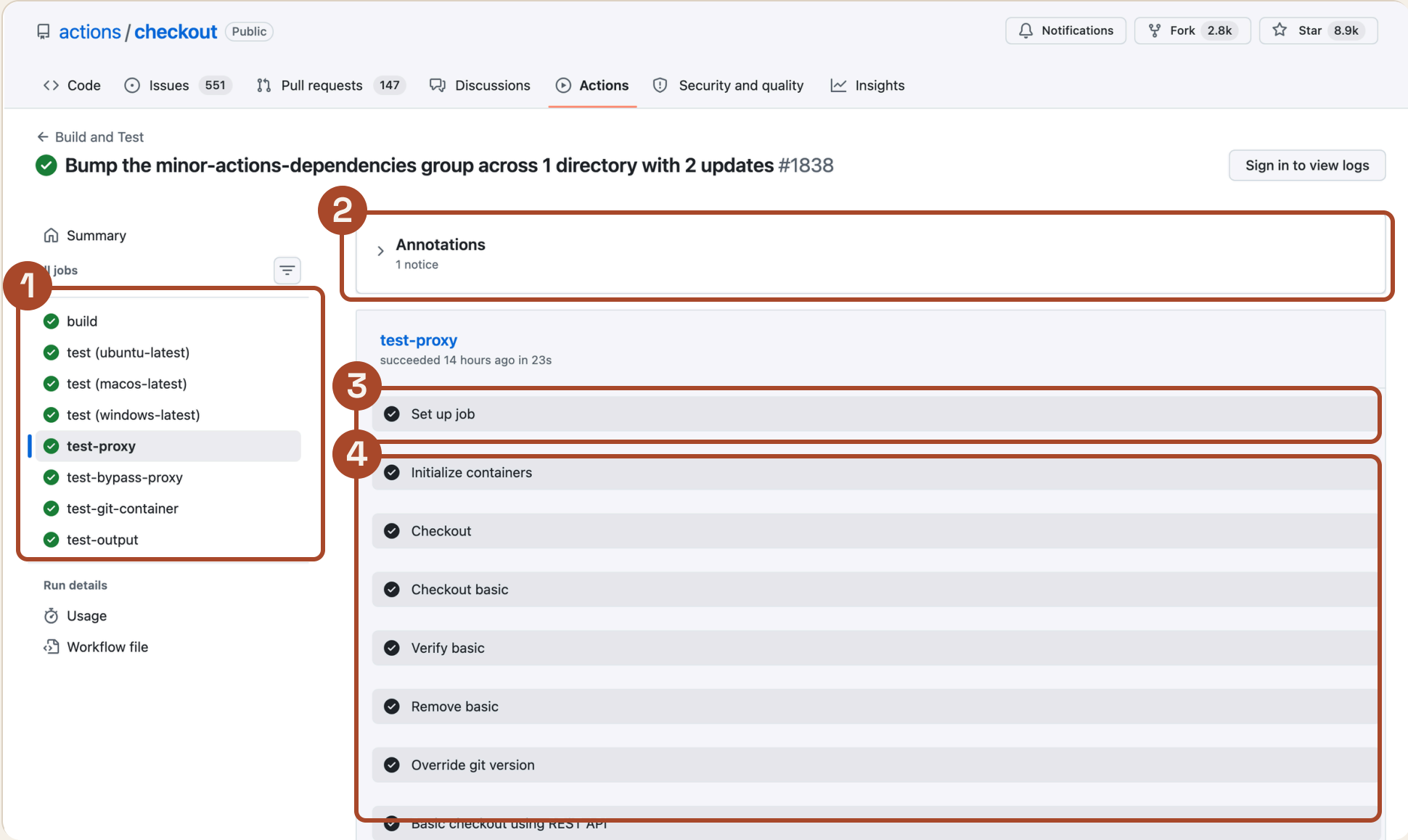
# One run, job by job



- 1 What this run is**  
The title, and the run number. #1838 is the 1,838th run of this workflow.
- 2 Trigger, status, duration**  
Who or what started it, whether it passed, how long it took, and how many artifacts it produced.
- 3 Every job, one row**  
build, then test three times. Those three are one matrix job expanded into one row per operating system.
- 4 The job graph**  
The same jobs drawn as a picture, with the timing of each. Arrows between boxes are **needs** relationships.

Screenshot: a Build and Test run on actions/checkout.

# Every step, in order, with its own log



- 1 The job list, matrix and all**  
build, then test three times, one per operating system. That is the matrix you wrote, expanded into one row each.
- 2 Annotations, before any log**  
GitHub lifts warnings and errors out of the logs and puts them at the top. Read this first, every time.
- 3 Set up job is GitHub's**  
You did not write it. It lists the runner image and every action version that was pulled. Start here when something is mysteriously missing.
- 4 Then your steps, in order**  
Checkout, then the work. Each row expands into its own log. Click the step name, not the job name.

Screenshot: one job from a run on actions/checkout.

# Commit often, and let the machine check every change

## The same checks, every time

No reviewer forgets to check the formatting, or whether somebody left a storage account public, at 6pm on a Friday.

## Caught before the merge

A problem found on a pull request costs minutes. The same problem found in production costs an incident report and a weekend.

## Evidence you did not have to write

Every run is logged with who, what, when and the result, and it is queryable. Your auditors will like this more than you do.

## What GitHub gives you for free

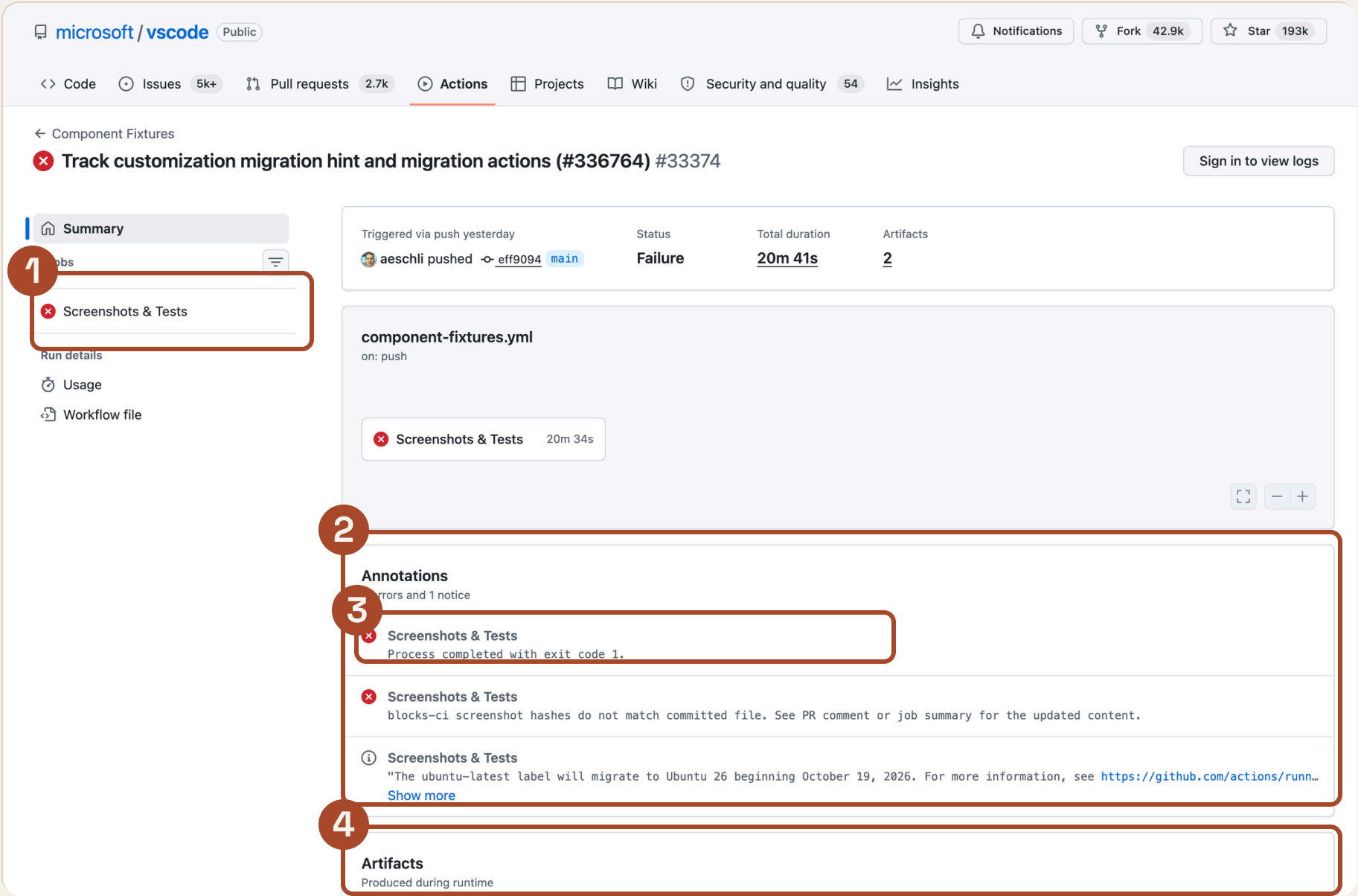
Results appear on the pull request as passed or failed. GitHub reads your code and suggests a starter workflow for the language it finds. Use it, change it, or ignore it and write your own.

## Our CI for tomorrow, in four checks

**fmt** is it formatted · **validate** does it make sense · **tflint** common mistakes · **Checkov** risky settings. That is exactly what you build in Lab 2.

WORKFLOWS • WHEN A RUN GOES RED

# Four places to look, in this order



Screenshot: a genuinely failing run on microsoft/vscode.

- 1 The job list, not the title**  
Find the red X in the sidebar. That names the job that broke. The title at the top just says something broke.
- 2 Annotations**  
GitHub pulls the errors out of the log for you. Read these before you open a single log file.
- 3 The real message**  
"Process completed with exit code 1" is the symptom. The cause is the annotation above it, or a few lines up in the log.
- 4 Artifacts survive a failure**  
If the upload step used `if: always()`. A failed run's report is usually the thing you actually need.

# Round 2: Workflows and pipelines

1. GitHub reads your workflow file from which commit?
2. Two jobs, no needs between them. Which runs first?
3. A matrix leg fails. What happens to the other legs by default?
4. Where do you look first when a run goes red?

# Checks on every pull request

## GOAL

Nobody can merge Terraform that is badly formatted, broken, or quietly insecure.

## DONE WHEN

A pull request cannot be merged until both jobs are green, and you have seen it stop you.

1. On a branch, create **terraform-ci.yml** with a **check** job and a **scan** job
2. Open a pull request and watch both jobs run side by side
3. Break the formatting on purpose. Read the failure. Fix it
4. Read the **#checkov:skip** lines in main.tf. Would your risk team accept those reasons?
5. Protect main: require a pull request and both checks. Then merge

# Variables and contexts

Where a workflow gets its facts, and what it does when something fails. A first look today, the full treatment tomorrow.

---

## YOU WILL BE ABLE TO

- Choose correctly between env, vars and secrets
- Read contexts and the default environment variables
- Control what runs after something fails
- Know what tomorrow adds: expressions, outputs, artifacts

## WATCH AND READ

- Docs: Variables
- Docs: Contexts
- Docs: Expressions, for tonight
- Docs: Store information in variables

# env, vars and secrets are not interchangeable

|             | env                     | vars (configuration variables)           | secrets                                  |
|-------------|-------------------------|------------------------------------------|------------------------------------------|
| Set where   | In the workflow file    | In Settings, at org, repo or environment | In Settings, at org, repo or environment |
| Shared by   | Only that workflow      | Many workflows                           | Many workflows                           |
| Read with   | env.NAME or \$NAME      | vars.NAME                                | secrets.NAME                             |
| In the logs | Shown                   | Shown, <b>not</b> masked                 | Masked                                   |
| Use it for  | Values for this one run | Settings, IDs, region names              | Passwords and tokens, nothing else       |

## Closest scope wins

Environment beats repository, repository beats organisation. When a value is not what you expect, check all three levels before you blame the workflow.

## Naming rules

Letters, digits and underscores only. Cannot start with a digit, and cannot start with **GITHUB\_**, that prefix is reserved.

## The limits

48 KB each. Up to 500 per repository, 1,000 per organisation, 100 per environment. If you are near those, something has gone wrong.

# Two ways to read the same fact

| Set on the runner for every job | Holds                               |
|---------------------------------|-------------------------------------|
| GITHUB_REF                      | The branch or tag that triggered it |
| GITHUB_SHA                      | The commit that triggered it        |
| GITHUB_REPOSITORY               | owner/repo                          |
| GITHUB_ACTOR                    | Who started it                      |
| GITHUB_WORKSPACE                | Where your code is checked out      |
| RUNNER_OS                       | Linux, Windows or macOS             |

```
- if: ${ github.ref == 'refs/heads/main' }
  run: echo "Deploying from $GITHUB_REF"
```

## The difference that matters

The **if** uses a context, because GitHub evaluates it **before** the job ever reaches a runner. The shell command uses the environment variable, because by then there is a machine to read it from.

## So the rule is

Deciding whether something runs: use a context. Inside a running command: either works, and the environment variable is usually tidier.

# Anything inside the double braces is an expression

| Context              | What is in it                             |
|----------------------|-------------------------------------------|
| github               | The event, repo, branch, actor, SHA       |
| env · vars · secrets | The variables and secrets you set         |
| inputs · matrix      | Run inputs, and the current matrix values |
| needs · steps · job  | Results from earlier jobs and steps       |
| runner · strategy    | The machine, and the matrix settings      |

## Not every context works everywhere

You cannot read **steps** before any step has run, and you cannot read **secrets** in a job-level **if**. The docs have the full availability table. When something evaluates to empty, that is usually why.

## Empty is not an error

An expression that cannot resolve becomes an empty string and carries on. No warning. That is why a typo in a context name shows up as a weirdly empty log line rather than a failure.

Anything that lands in a **run:** block becomes part of the script before the shell sees it. Remember that sentence, it is the whole of tomorrow's script injection session.

# Deciding what runs after something fails

```
- name: Run the tests
  run: ./run-tests.sh

- name: Tell the team
  if: failure()
  run: echo "Failed on ${github.ref_name}"

- name: Upload the report anyway
  if: always()
  uses: actions/upload-artifact@<SHA>
```

| Function    | True when                                         |
|-------------|---------------------------------------------------|
| success()   | Everything before it passed. This is the default. |
| failure()   | Something before it failed                        |
| always()    | Always, even when the run was cancelled           |
| cancelled() | The run was cancelled                             |

**If you write no status function**

GitHub adds **success()** for you. That is why a step after a failure normally does not run, without you writing anything.

**Careful with always()**

It runs even after a cancel, so an **always()** step that takes ten minutes makes cancelling useless. Use it for cleanup and reports, not work.

# Round 3: Variables and expressions

1. Which of env, vars and secrets is masked in the logs?
2. Which context tells a workflow which branch it is running on?
3. What does a misspelled context name evaluate to?
4. Which status function still runs after a cancel?

# Make a workflow think

## GOAL

Use variables, contexts, expressions and status functions in one workflow, and watch the decisions happen.

## DONE WHEN

The summary shows your team name, and in the failing run the failure step ran while the success step did not.

1. Find the repository variable **TEAM\_NAME** in Settings, then print it with **vars.TEAM\_NAME**
2. Add a **choice** input with a **fail** option, and a step that fails when it is chosen
3. Add one step with **if: failure()** and one with **if: always()**
4. Write a table of context values to **\$GITHUB\_STEP\_SUMMARY**
5. Run it twice, once passing and once failing. Compare which steps ran

# Draw the pipeline before you build it

Open **excalidraw.com**. As a team, draw tomorrow's Terraform pipeline. Boxes and arrows are enough. Your drawing has to answer all eight of these.

1. Where does a change start?
2. Every automatic check, and what each one stops
3. Where does a human see the plan?
4. What happens on merge?

1. Who approves production, and what do they read first?
2. How does the pipeline log in to Azure?
3. Where does Terraform state live, and what stops two changes at once?
4. What happens if somebody changes Azure by hand?

## 20 min

Draw it as a team.

## 2 min each

Present. Other teams ask "what if..."

## Keep it

Save it as  
**docs/pipeline.excalidraw**  
through a pull request.

END OF DAY 1

# What you can do now, and what is next

## TODAY

- Explain every building block of GitHub Actions
- Write triggers, jobs, matrices and choose a runner
- Use an action from the Marketplace without taking a risk
- Build CI that blocks a merge, and read a red run
- Use variables, secrets and contexts

## TOMORROW

- Expressions, job outputs, passing data between jobs
- Artifacts, caching and concurrency
- Environments, approvals, and reuse across repos
- Log in to Azure with OIDC, then build the pipeline
- Security: pinning, token permissions, script injection

If you want more tonight: Episode 3 covers Part 2 again in 67 minutes, and the docs page for Workflows is the one worth bookmarking.